



ROBOCUPJUNIOR RESCUE (LINE/MAZE) 2026

TEAM DESCRIPTION PAPER

Team FATEN (L11)



Kent Nakai, Yuna Tamura, Hampus Mollgård, Rie Nakai

Abstract

FATEN is a RoboCupJunior Rescue Line robot designed with reliability as the main goal. The robot was not designed only to complete one good run. It was designed to make the same behavior repeat even when the field condition changes. For this reason, our design focused on three main ideas: stable contact with the ground, separated sensing systems, and modular decision making.

Mechanically, the robot uses a flexible enhanced-acrylic suspension plate. This suspension lets the wheels keep contact with the field even when the ground height changes. The robot is about 140 mm by 180 mm and weighs about 800 g, with mainly laser-cut acrylic parts. It uses four STS3032 serial servos, custom 3D-printed wheels with silicone molded tires, and an arm mechanism that can pick up and hold two victims.

Electronically and computationally, the robot is separated into several clear roles. The Teensy 4.1 handles the main control, motor output, state logic, mapping, and sensors. The XIAO ESP32-S3 with an OV2640 camera handles line and feature detection. The K230D runs a YOLOv8n model for victim detection in the evacuation zone. This separation makes debugging easier because each controller has a clear responsibility.

Testing showed that the robot could run line following and intersection behavior for 40 minutes continuously without a mistake, and the suspension could pass over a 60 mm height difference. The evacuation-zone AI system is still being improved, especially the ONNX to kmodel conversion process, but the current design gives a stable base for the USA 2026 competition.

1. Introduction

a. Team

Kent Nakai - Head developer

This is my tenth year in RoboCup. When I started at nine, all I wanted was to make a robot move forward. From there, competing has driven me to build robots that adapt to their environment. Four years ago I moved to the US, built teams from scratch at three different schools, and got hooked on building something great as a team. We won U.S. Nationals three years in a row, but with many fails. Last year, our robot couldn't even drive straight through a whole competition. And the thing that grew the most through all of it wasn't technical. It was how I understand other people, and myself.

This year is built on all of that. Hampus, who I met in the Netherlands, friends I used to compete with in Japan, and friends I made here in the US, we came together to build one robot that balances everything: hardware, software, and funding.

Yuna Tamura - Hardware and Management

I focus on hardware development, while also handling most of the managerial work, as well as taking responsibility for sponsorship and public relations. Hardware support is one of the roles I'm most proud of — I genuinely enjoy building, problem-solving, and working hands-on with the team. Beyond the technical work, I care a lot about supporting the team in every way I can. Sometimes that means coordinating logistics, designing our team apparel, or photographing our activities so we can share our progress with others. I've competed in RoboCupJunior in Japan, and in 2021 I placed 2nd in Rescue Line Primary at RCAP Nagoya. That experience shaped how I approach teamwork today: combining technical development with the behind-the-scenes work that keeps everything running smoothly.

Hampus Mollgård - Mentor and advisor

With a long background as team leader of a Swedish robotics team, I first met Team FATEN as a competing team in Eindhoven 2024. Since then, we have stayed in contact and developed this international collaboration. Having already competed internationally for two years in this league, I am no longer eligible as a participant. My role with Team FATEN is therefore as a mentor and advisor, supporting them with experience, strategy, documentation, and technical discussions, while the robot remains the responsibility of the competing team members.

Joshua Enelamah - Advisor

I competed alongside Kent last season, and I saw first-hand how many ideas this team wanted to fit onto a single robot. That ambition excited me to want to keep working with him and FATEN. This year I continue that collaboration from last year mainly on Electronics and Software Ideas, so the rest of the team can push those ideas as far as the hardware will allow.

2. Project Planning

a. Overall Project Plan

The goal of Team FATEN in 2026 is to build a fully capable Rescue Line robot that can compete at a high international level. Beyond raw competition performance, two focuses shape how we work. First, as a geographically distributed team of future engineers, we use real-world engineering practices and collaboration techniques that matter far beyond RoboCup, so we can communicate smoothly and build a robot that moves with true intent. Second, we keep a fast iteration time and a separate platform for testing every new concept quickly on real hardware. Those two focuses decide *how* we work. *What* we build toward comes from Rescue Line itself, where we set two technical priorities: reproducible line following on slopes, and robust victim detection. From these focuses and priorities together, we defined the following requirements for the robot:

- The robot must always drive with intent, with active compensation adapting to different types of terrain.
- The robot must always keep stable contact with the ground on slopes, bumps, and uneven tiles.
- The robot should make no major decision based on a single frame of information.
- The robot must be able to calibrate at the competition arena, so sensor values such as the RGB camera channels can be mapped linearly and adapt to the lighting condition.
- Minimize any software solution that can be solved by hardware.
- The software must be structured as a state machine with modular functions, keeping the control logic clear to follow and simple to scale with new methods.

Roles: [K] Kent [Y] Yuna [J] Joshua (advisor) [H] Hampus (mentor)

Assignment logic: software (control / AI / navigation) → [K]; electronics → [J,K]; mechanical & hardware build → [K,Y]; sponsorship → [Y]; planning & vision → [K]; mentoring & review → [H].

Our Workflow. Because the team is spread across countries, our plan needed to incorporate a clear way to work together. We used WhatsApp for fast communication, GitHub for code, and Notion for documentation, plus online meetings and in-person work during school breaks. Hampus reviews our strategy and documentation at each gate. But our most important performance feedback, and review gate is always the robot itself: if it could not repeat a behavior on the field, the design was treated as unfinished, even if the part worked in isolation.

Adjusting the Plan. The first plan was to finish the core hardware and electronics early, then spend most of the time on advanced software such as navigation and AI detection. During development we learned that the hardest part was not building each part separately, but getting many systems to actually work together as one robot, as one team. So the plan shifted toward making the base behavior fully reproducible first. Instead of adding features quickly, we focused on making the main movement, sensing, and controller structure stable, and only then scaled up.

Ph.	Period	Lead	Key milestones	Gate (review)
P1	Jun–Aug 2025	[J,K,H]	Audit 2025 robot; freeze carry-over subsystems; set the two goals [→slope] [→AI]; write rule-traceable requirements.	G1 Every requirement maps to a rule with a measurable target.
P2	Sep–Dec 2025	[K,J,Y]	SW: test slope control, compare PID variants, trial line-sensing. HW: PCB schematic + layout [J,K]; sponsorship [Y]; train + compare detection models.	G2 One approach per goal (with data); PCB fab-ready; detection model chosen.
P3	Jan–Mar 2026	[K,J,Y]	Chassis + suspension [K,Y]; PCB, wiring, inter-controller comms [J,K]; modular SW + state machine; flat follow, intersections, slope (case-split); deploy detection on K230.	G3 All boards talk, drives stably (manual). Every task type cleared at basic level.
P4	Apr–May 2026	[K]	Generalize: one non-linear PID controller for any surface; evac-zone full-coverage sweep; robust detection in complex terrain; merge flat + slope.	G4 One controller passes flat + slope (no case-split); sweep reliable; detection robust.
P5	May–Jun 2026	[All]	Win USA Nationals (qualify for Worlds); failure analysis → fix list [K,H]; training camp; full-system reliability runs; rehearse on-site calibration.	G5 Qualified; scored run repeatable; failure rate below target.
P6	from Jun 2026	[All]	World Championship, Incheon: on-site calibration, compete + log per-run data, post-event review.	—

Table 1: Project timeline. Two goals tracked across phases: [→slope] generalized slope line-following, [→AI] robust victim detection. Gates are review points that must pass before the next phase.

b. Integration Plan

Our integration follows one rule: each subsystem owns a single responsibility. If all sensing, AI, motor control, and decision making sat in one controller, a single line-following failure could come from the camera, communication, PID, motor timing, or state logic at once, and we would not know where to look. To keep failures findable, we split the system into separate controllers with defined boundaries.

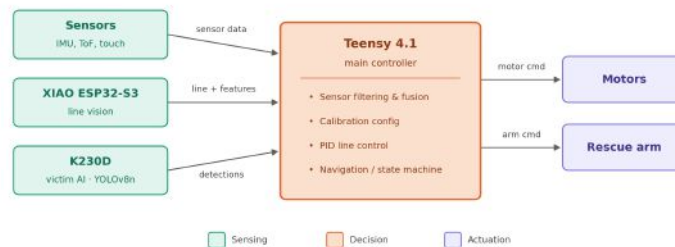


Figure 1: Robot integration diagram

The **Teensy 4.1** works as the main controller. It receives line and feature information from the XIAO ESP32-S3, receives AI detection results from the K230D, reads sensors such as the MPU6050 IMU, VL53L7CX time-of-flight sensor, touch sensors, and custom conductivity sensors, and then controls the motors and rescue mechanism.

The **XIAO ESP32-S3** works as the line-vision controller. It uses an OV2640 camera to detect the line,

intersections, red tape, silver tape, and gaps. The XIAO sends compact information to the Teensy, such as the line-center position and feature command. This keeps the Teensy focused on decision making instead of raw image processing.

The **K230D** works as the AI vision controller for the evacuation zone. It runs an object-detection model such as YOLOv8n or MobileNet-SSDv2-lite to detect victims and evacuation-zone objects. The K230D does not control the robot directly. It only provides detection information to the Teensy, and the Teensy decides the action.

3. Hardware

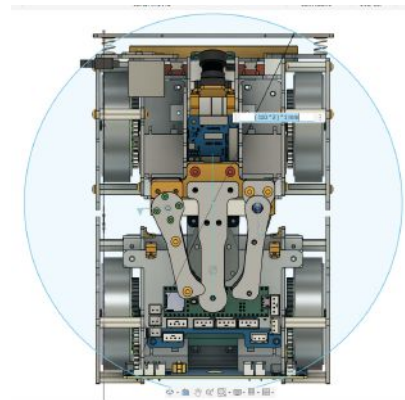
Two ideas drove the mechanical design. First, we decided the core shape of the robot so it fits the field and turns the way we want. Second, we made sure we stay in full control of the robot at all times.

a. Mechanical Design – Structure and Diagrams

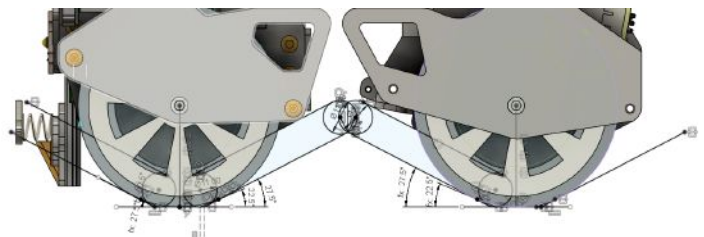
Body Shape and Dimensions

The chassis shape was decided from the driving behavior first, not only from component placement. In early slope-turning tests, a center-based pivot caused the four-wheel-drive chassis to scrub sideways during spin turns, especially on inclined tiles. Therefore, we placed the effective rotational axis slightly behind the geometric center of the robot. With a rear-shifted pivot, the rear side stays more planted while the front of the robot swings back toward the line, which improved recovery after slope corners and reduced downhill sliding.

After fixing the pivot position, the final body size was checked in CAD using two constraints. First, the top-view envelope was limited to a 220 mm diameter turning circle, while still leaving space for the front rescue arm, camera, and sensors. This led to the final chassis size of approximately **140 mm × 180 mm**. Second, the side profile was checked against bumps and a 27.5° clearance line, so the lower frame would not catch on field obstacles or slope transitions.



(a) Top-view size envelope. The 220 mm circle shows the swept area during a tight turn. The final chassis size is about 140 mm × 180 mm.



(b) Side clearance check. The frame profile was shaped so the chassis clears bumps and slope transitions without scraping.

Figure 2: Mechanical constraints used to determine the final chassis shape: rear-shifted rotational axis for stable slope turning, compact top-view envelope, and side clearance for bumps and slopes.

Stability and Control Layout

After the body envelope was fixed, the next design target was full control of the robot. For us, "control" means that the robot should turn, climb, and recover in a predictable way even when the field surface is uneven.

This connects directly to the rear-shifted rotational axis described above. The rotational axis decides how the robot turns, but the center of mass decides how strongly the wheels load the field during that turn. We therefore placed heavy parts, especially the motors and battery, low and close to the drive base, while keeping the XY center of mass balanced around the selected pivot area. This reduced diagonal weight transfer during spin turns and made the same steering command produce a more repeatable motion on flat tiles, bumps, and slopes.

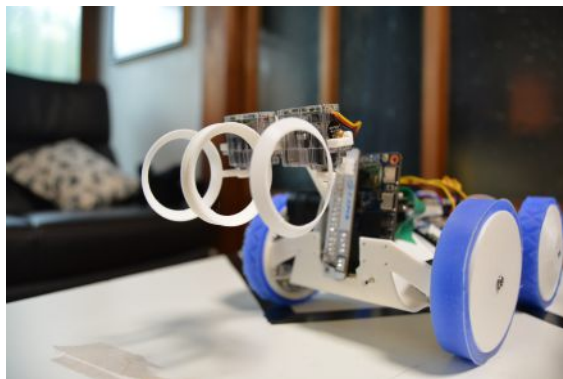
The chassis also uses a central rotation joint in the suspension. This lets the left and right sides of the drivetrain follow small height differences without lifting one wheel off the field. The result is more stable traction: instead of the robot balancing on three wheels, all four wheels keep contributing to drive and braking.

Front Modules: Arm and Debris Remover

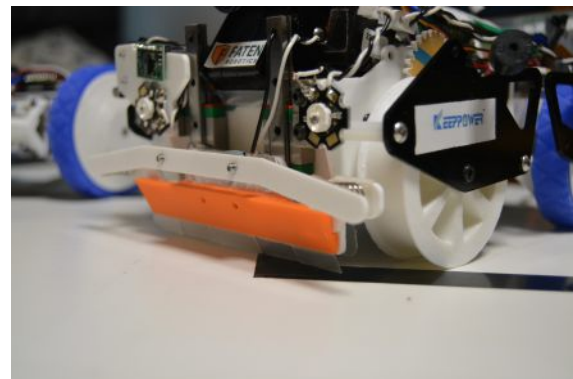
The front of the robot had to satisfy two requirements at the same time: it needed space for the rescue arm, but it also had to protect the line-following camera from debris. Both modules were therefore designed inside the same body envelope and checked against the side-clearance profile in Fig. 2.

The rescue arm can hold two victims. This is useful because the robot may find a black ball before collecting both silver balls. With two holding positions, the robot can change the loading order instead of leaving a victim behind. To reduce the effect on the front mass, the claw was originally lasercut, and later, on our prototyping base, printed thinly (**0.8 mm**, two passes of a 0.4 mm nozzle), which was light but still strong enough to hold the balls during movement.

The debris remover is mounted at the front, just before the camera view cone. It moves on LWL7B linear guide rails and is actuated mainly by its own weight. This makes it passive and reliable: on flat ground it stays low enough to block debris from entering the camera area, but when it meets bumps or slope transitions, it slides upward so the robot can pass without the frame catching.



(a) Two-ball arm layout. The loading order can be changed depending on which victim is found first.



(b) Gravity-following debris remover. The linear slide keeps it low on flat ground and lets it rise over bumps and slopes.

Figure 3: Front mechanical modules designed within the same chassis envelope: a lightweight two-ball arm and a passive debris remover that protects the camera path.

b. Innovative Sub-module Designs and Verification

IR Camera for LED-Robust Victim Detection

The 2026 rules add evacuation-zone LED lights mounted perpendicular to the field, so visible-light victim detection became less reliable. To reduce this risk, we built an IR camera by removing the RGB filter and adding an 850 nm bandpass filter.

The reason is that the lighting problem and the victim-detection problem move in opposite directions under IR. Competition LEDs emit much less IR than visible light, so their effect on the camera is reduced. At the same time, silver victims reflect strongly under IR while black victims absorb more light, which increases contrast for detection.

We first tested 940 nm, but it interfered with the ToF sensors. After that, we switched to 850 nm. The robot also carries its own IR lighting, so the camera view depends more on our controlled illumination than on field lighting or camera flashes. We verified the setup by testing the AI model with the 850 nm camera image and by comparing the camera view before and after improving the diffuser.

Manufacturing Methods for Fast Iteration

Most structural parts are made from 3D-printed PLA, laser-cut plates, molded silicone, and hand-finished surfaces. Two methods were especially useful during development.

First, we used friction welding for PLA prototypes. A rotary tool spins a small piece of plastic, melting it into the joint between two printed parts. This is faster than reprinting a full part and, in our tests, stronger and cleaner than gluing for many prototype repairs.

Second, we made our own LED diffuser for the camera box. The camera box was printed in white plastic and sanded so the inside surface scattered light. Then we molded silicone mixed with baby powder to create a soft diffuser. We tested several fillers, including food coloring and flour, but baby powder gave the most even camera illumination.



(a) Friction welding PLA parts with a rotary tool for fast repairs and prototype changes.



(b) Diffuser development using a sanded white camera box and molded baby-powder silicone.

Figure 4: Development methods used to shorten mechanical iteration while improving the reliability of the vision system.

The diffuser was verified directly with the camera image. Before the diffuser, the LEDs created bright hot spots near the sides of the image. After adding the diffuser and sanding the camera box, the light was spread more evenly across the view, giving a more consistent input for victim detection (Fig. 5).

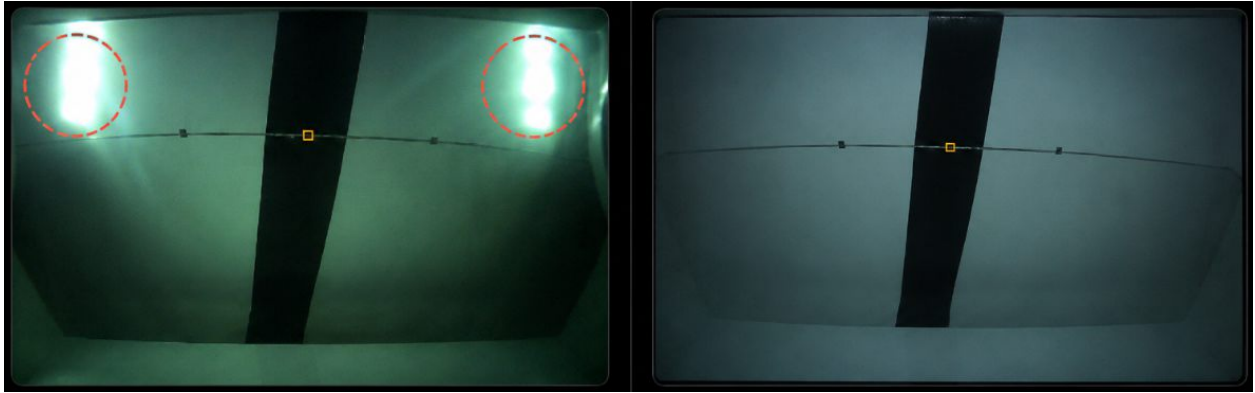


Figure 5: Camera view before and after adding the diffuser and sanding the camera box. Left: without diffuser, the LEDs create strong hot spots and uneven illumination. Right: with the diffuser, the light is spread more evenly across the camera view.

4. Electronics

The electronic system is built around a Teensy 4.1 main controller, with two co-processors that take the vision load off it: the XIAO ESP32-S3 for line sensing and the CanMV K230D for evacuation-zone AI. Sensors and actuators connect over clearly separated buses, fed by a custom power stage (Figure 6).

The three processors are chosen so each runs stable within limits. Heavy work like image processing and neural-network inference can easily disrupt real time timing control so its keep it off the Teensy. The XIAO handles the lightweight line vision, and the K230D, a chip optimised for neural-network computation, runs the detection model on its AI accelerator. That leaves the Teensy with a light load, so it can hold its control loop at a fixed rate and no single board is ever pushed past what it can finish in time.

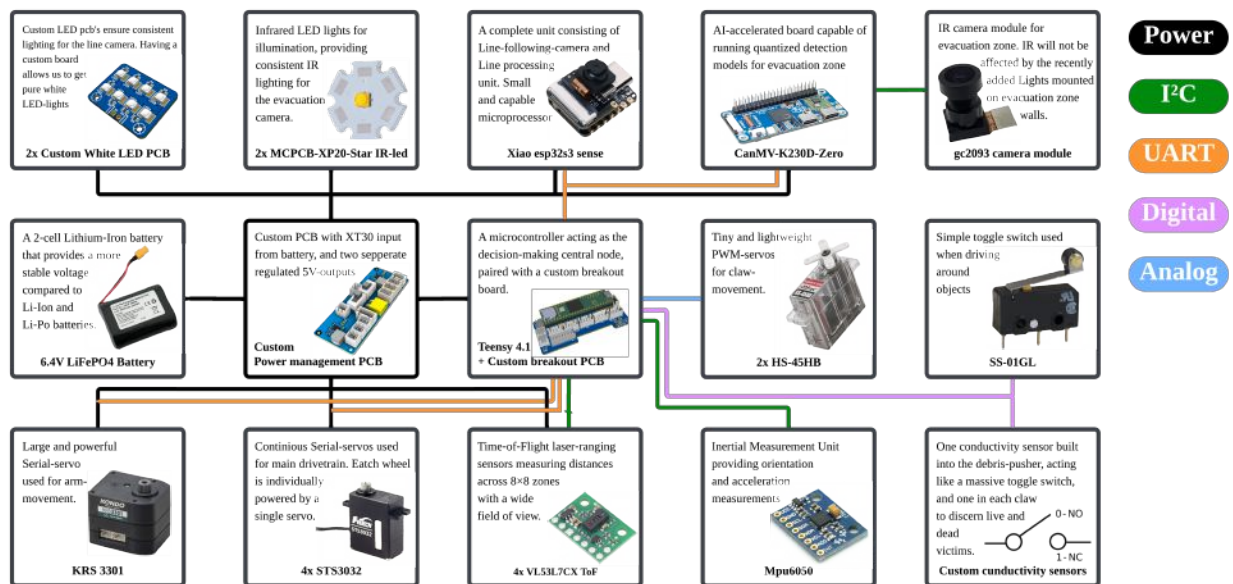


Figure 6: Electronic system map, colour-coded by bus (power, I²C, UART, digital, analog).

Custom PCBs. We fix last year's noise in the layout itself. The motor power and the logic power sit on their own copper, and both grounds come back to one star point by the battery input, so the motor's current spikes do not move the logic ground. The bulk and decoupling caps sit right at the regulator and at each chip's power pin, so the switching current stays in a small loop and does not radiate into the sensor lines. For heat, the regulator pad ties into a large GND copper pour through a grid of thermal vias, so the ground plane spreads the heat out instead of it building on the chip. With this the supply stays stable and the controller does the same thing every run.

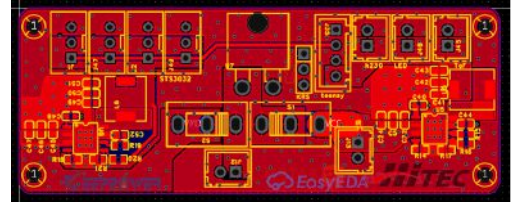


Figure 7: Custom power PCB: a stable supply for the whole robot.(via stitches, star Ground, heat dissipation design)

Half-duplex servo driver. The KRS3301 and STS3032 are half-duplex serial servos: each uses one bidirectional data line at 5 V logic, while the Teensy has separate RX and TX pins at 3.3 V. We bridge both gaps with a 74HCT126 tristate buffer. When sending, the HCT buffer reads the Teensy's 3.3 V TX as a valid HIGH and drives the line at a clean 5 V, and its tristate enable releases the line so the servo can reply without contention. When receiving, the servo's 5 V output is dropped to 3.3 V through a voltage divider before it reaches the Teensy. One UART now drives both servos cleanly, level-shifted in each direction.

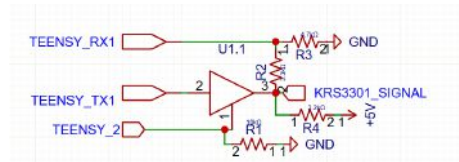


Figure 8: Tristate-buffer interface for the 5 V serial servos.

Why LiFe Battery. We use a 2S 4800 mAh LiFePO₄ pack. LiFePO₄ holds a nearly constant voltage across most of its discharge before dropping off, unlike Li-ion which sags steadily. A constant voltage means every part sees the same conditions from the first run to the last helping reproducibility.

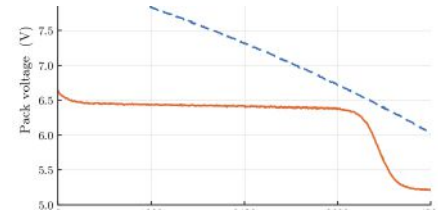


Figure 9: LiFe battery used for stable robot operation.

Quality assurance for the PCB is not a one-shot design. It got better over years, one iteration at a time, and each version shows us what holds up and what does not. We keep what survives real testing on the robot, so the more we run it the more we trust it. Nothing is perfect though, so we always keep spare PCBs and can swap a board fast if one ever burns.

5. Software

a. Architecture Design

The software runs on three boards in two languages. The XIAO ESP32-S3 runs the camera vision, the K230D runs the AI model, and the Teensy 4.1 is the conductor that decides what the robot does. The Teensy and the XIAO are both Arduino (C/C++); the K230D is MicroPython. The two co-processors do the heavy image processing, so the Teensy is left to think and act.

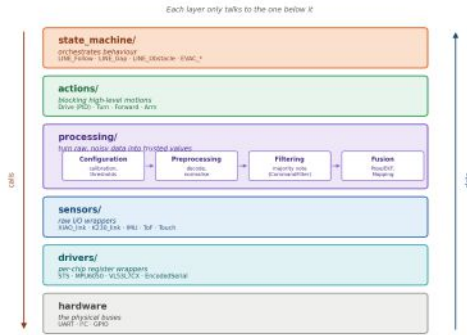


Figure 10: The Teensy code is a stack of layers. Each layer only talks to the one below it: behaviour calls down, sensor data flows up.

Main loop. Every cycle does the same things in order: pump the sensor I/O, run processing (decode, filter, fuse), then run the active state. The loop runs at a fixed rate, so the control stays predictable no matter what the vision boards are doing.

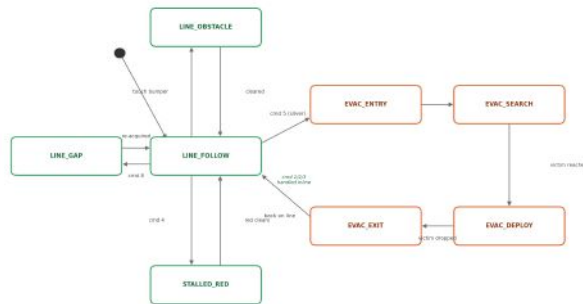


Figure 11: The behaviour is a state machine. It lives in LINE_FOLLOW and branches out on events, with the evacuation zone as its own chain.

Inside the modules. The work under each layer is real, not a stub: a command from the XIAO is not trusted until a 15-frame majority vote agrees on it (CommandFilter), the K230 boxes are checked with a header and an XOR before use, and the line error drives a PID that mixes into four wheels through a 9 ms ISR. We keep these details in the modules so the state machine stays easy to read.

b. Innovative Software Solutions

6. Performance Evaluation

The performance evaluation focused on the two main goals of the robot: clean slope driving and robust evacuation-zone detection. Slope driving was evaluated through the mechanical layout, suspension, and terrain-biased PID controller. Therefore, this section focuses on the remaining high-risk system: the evacuation-zone AI model. For the AI model, we did not evaluate only training accuracy. The real question was whether a detector could survive the full embedded pipeline: image collection, preprocessing, training, ONNX export, quantization, kmodel compilation, simulator testing, and deployment on the K230D. We

Layered structure. On the Teensy, the code is split into layers, and each layer mostly talks to the one below it. The state_machine/ at the top decides the behaviour. Under it, actions/ contains the blocking motions, such as Drive PID, Turn, and the arm.

processing/ is where raw data becomes data we can trust, in four steps. Configuration sets the calibration and thresholds, so the same code adapts to the arena lighting. Preprocessing decodes and normalises the raw bytes from the XIAO and the K230D. Filtering rejects noise; for example, a command is only accepted once a 15-frame majority vote agrees on it. Fusion combines the clean values into pose and a map the state machine can use. Nothing upstream is trusted until it has passed through these steps.

State machine. The behaviour starts in LINE_FOLLOW, where most of the run happens. Events switch it out: front touch goes to LINE_OBSTACLE, lost line (cmd 8) to LINE_GAP, red tile (cmd 4) to STALLED_RED, and silver tape (cmd 5) starts the evacuation chain: EVAC_ENTRY → EVAC_SEARCH → EVAC_DEPLOY → EVAC_EXIT. Simple line actions, such as U-turn and green left/right turns (cmd 1/2/3), are handled inside LINE_FOLLOW because they act but do not change the main state. This keeps each state small, so fixing one behaviour does not affect the others.

collected raw RGB images from the robot and generated several Python-processed dataset variants, including raw RGB, shadow-separated images, and CLAHE / histogram-equalized images. These variants were used to check whether image preprocessing improved victim contrast before training. We then trained and compared several detector families, including MobileNet-SSD-style models, AnchorBaseDet-style models, and YOLOv8n. Each trained model was exported to ONNX and passed through the K230D nncase pipeline. The conversion process was automated so that different quantization and compiler options could be tested repeatedly. The results were then summarized into an HTML comparison report, allowing us to compare not only model size and recall, but also whether the highest-scoring detection remained reliable after quantization. *Data disclosure.* The comparison used 50 validation images and 78 labeled victim objects. Because this is still a limited dataset, we used the results mainly for engineering model selection, especially quantization behavior and single-best detection reliability, rather than as a final statistical claim of competition accuracy.



Model	Size (KB)	Recall	Precision	F1	Avg_BSI
MobileNet-SSD	1.2	0.85	0.75	0.80	0.85
AnchorBaseDet	1.5	0.88	0.78	0.83	0.88
YOLOv8n	1.8	0.90	0.80	0.85	0.90

Figure 12: K230D AI model evaluation summary. The comparison shows that smaller models were not automatically better after quantization. The final selection prioritized reliable single-best detection after conversion to a K230D `kmodel`.

The key result was that the smallest detector was not the best deployable detector. Some AnchorBaseDet models were compact, but after uint8 PTQ their classifier output collapsed, so the highest-confidence box became unreliable. This failure matters because the K230D sends a compact detection result to the Teensy; if the highest-scoring box is wrong, the robot may move toward the wrong victim even when another correct box exists lower in the output list. Because of this, we evaluated `avg_bsi`, the average IoU of the highest-scoring same-class box, in addition to normal recall. This metric better matches the robot's real decision path than recall alone. YOLOv8n with int16 activations and uint8 weights gave the best deployable balance, preserving both localization and single-best ranking after quantization while staying much smaller than the float model. This evaluation changed our design rule for AI: the chosen model is not the model with the highest PC-side training result, and not simply the smallest model. It is the smallest model that keeps reliable detection after the full embedded deployment pipeline.

7. Conclusion

FATEN's 2026 Rescue Line robot was designed around consistency. The main challenge was not only making a robot that can complete each task once. The harder challenge was making a robot that can repeat the task when the field condition changes.

To solve this, the robot uses a flexible suspension for stable ground contact, custom wheels for traction, modular PCBs for cleaner electronics, and separated controllers with clear roles. The Teensy handles main decision making, the XIAO handles line vision, and the K230D handles AI detection. This structure made the robot easier to debug because each subsystem has a clear responsibility.

Testing showed that the robot could run line following and intersection behavior for 40 minutes without a mistake, and the suspension could pass over a 60 mm height difference. At the same time, the AI deployment process showed that advanced features are not finished just because the model exists. The conversion from ONNX to `kmodel` was a major engineering challenge.

Overall, the robot provides a stable base for the USA 2026 competition. The next step is to keep improving evacuation-zone rescue behavior and full-system reliability.

References

- [1] RoboCupJunior Rescue Technical Committee. *RoboCupJunior Rescue Line Rules 2026*.
- [2] PJRC. *Teensy 4.1 Development Board Documentation*.
- [3] Seeed Studio. *XIAO ESP32-S3 Sense Documentation*.
- [4] CEVA / Hillcrest Labs. *BNO085 9-Axis Sensor Documentation*.
- [5] STMicroelectronics. *VL53L7CX Time-of-Flight Sensor Datasheet*.
- [6] Ultralytics. *YOLOv8 Documentation*.
- [7] Kendryte / nncase Project. *nncase Model Conversion Documentation*.